

Arquitectura modular de repositorios

Equipo de Inteligencia Artificial



Creado por
Javier Curto Hernández

Actualizado en febrero de 2026

CONTENIDOS

1. Introducción	1
2. Estructura de repositorios	2
2.1 Archivos base (en la raíz)	3
2.2 config/ (configuración estática)	4
2.3 data/ (persistencia y estado)	4
2.4 notebooks/ (capa de experimentación)	5
2.5 shared/ (librerías internas)	5
2.6 src/ (lógica central)	6
2.7 api/ (capa REST)	6
2.8 demo/ (capa visual)	7
2.9 docker/ (contenedorización)	7
2.10 scripts/ (ejecución manual / CLI)	7
2.11 tests/ (calidad)	7
3. Ciclo de vida y PRs	9
3.1 Estructura de Ramas	9
3.2 Ciclo de Vida	9
3.3 Política de “Limpieza Trimestral” (Auditoría)	10

1. Introducción

Este documento sirve como guía para la sesión técnica sobre **arquitectura modular de repositorios** aplicada a proyectos de Machine Learning y sistemas con LLM/RAG. Partimos de una realidad habitual en equipos de datos: gran parte del trabajo ocurre en **notebooks/** y, cuando el proyecto madura, aparece la necesidad de convertir prototipos en componentes mantenibles y desplegables.

El objetivo de esta sesión introductoria es entender por qué una estructura de repositorio bien definida mejora la **reproducibilidad**, la **colaboración** y la **puesta en producción** (API, demo, persistencia y tests). A lo largo del documento se presenta una estructura de referencia y se explica su propósito carpeta por carpeta, de forma que el equipo pueda adoptar un lenguaje común, reducir fricción en el onboarding y facilitar la revisión de código.

Estándar obligatorio del equipo. Esta arquitectura de repositorios no es opcional: será la estructura mínima exigida para nuevos desarrollos y para la evolución de proyectos existentes. El objetivo es que cualquier persona del equipo pueda localizar rápidamente la lógica central, los datos persistentes, la configuración, los puntos de entrada de entrenamiento/inferencia, y los elementos necesarios para desplegar y testear.

En las siguientes sesiones se complementará esta visión con ejemplos prácticos. El objetivo es proveer al equipo de herramientas y patrones que nos permitan gestionar de forma completa el ciclo de vida de nuestros desarrollos: desde la experimentación en notebooks, pasando por la implementación en **src/**, su exposición en **api/**, el despliegue con **docker-compose**, hasta el aseguramiento de calidad con **tests/**.

2. Estructura de repositorios

```
.
|-- requirements.txt
|-- README.md
|-- .env                # Variables de entorno
|-- .env.example        # Plantilla de variables de entorno (se versiona)
|-- .gitignore          # Archivos y carpetas ignorados por Git
|-- .venv/              # Entorno virtual local (no versionar)
|-- docker-compose.yaml # Orquestación de servicios
|-- pytest.ini          # Configuración de tests
|-- config/             # Configuración estática
|-- data/               # TODO lo que debe persistir (Volumen Docker)
|   |-- models/         # Modelos/artefactos y métricas
|   |-- files/          # Archivos planos
|   |-- bdd/            # Bases de datos
|   |-- mlflow/         # Tracking (runs, artifacts) si adoptamos MLflow
|-- shared/             # Librerías internas compartidas
|-- src/                # lógica central (Producto)
|   |-- __init__.py
|   |-- modules/
|       |-- [modulo_ml_clasico]/ # EJ: Forecasting
|       |   |-- __init__.py
|       |   |-- entrypoint.py    # INTERFAZ PÚBLICA (train/predict)
|       |-- [modulo_agente_llm]/ # EJ: Chatbot RAG
|       |   |-- __init__.py
|       |   |-- entrypoint.py    # INTERFAZ PÚBLICA (chat/server)
|-- api/                # Capa REST (FastAPI)
|   |-- main.py
|   |-- routers/        # Endpoints que llaman a src.modules...
|-- demo/               # Capa Visual (Streamlit)
|   |-- app.py
|   |-- pages/
|-- docker/             # Dockerfiles / configs de contenedores
|   |-- api/
|       |-- Dockerfile
|   |-- demo/
|       |-- Dockerfile
|-- scripts/            # Triggers manuales (CLI)
|-- notebooks/          # Sandbox (EDA / experiments / evaluation)
-- tests/               # Aseguramiento de Calidad
```

2.1 Archivos base (en la raíz)

Estos archivos definen el entorno, la orquestación y las reglas del juego del repositorio.

- **README.md**: Documentación maestra. Debe explicar la arquitectura, pasos de instalación, uso de scripts y cómo levantar los servicios.
- **requirements.txt**: Listado de librerías necesarias. Garantiza que el entorno sea reproducible en local y en Docker.
 - **Generación**: Se recomienda generar este archivo desde un entorno virtual limpio.
 - **Buenas prácticas**: Debe contener solo las librerías necesarias para el producto final (ej. `fastapi`, `scikit-learn`, `openai`). Librerías de desarrollo como `jupyter` o `pytest` deberían ir en un archivo aparte (`requirements-dev.txt`) para no engordar la imagen de Docker de producción.
 - **UV en vez de conda**: En caso de trabajar con UV el archivo de `requirements.txt` se sustituiría por los archivos `pyproject.toml` y `uv.lock`
- **.gitignore**: Lista de exclusión de Git. **Crítico** para evitar subir secretos (`.env`), archivos pesados (`data/`), cachés (`__pycache__`) y **entornos virtuales** (`.venv/`, `venv/`).
- **.env**: Archivo de configuración local para **secretos** (API keys, passwords de BBDD) y variables de entorno. **Importante incluir en .gitignore**.
- **.env.example**: Plantilla del archivo `.env`. **Este sí se sincroniza con GitHub**. Debe incluir todas las variables necesarias (con valores de ejemplo o vacíos) para que cualquier miembro del equipo pueda crear su `.env` local rápidamente.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de docker y testing

- **.venv/**: Entorno virtual del proyecto. Debe estar en `.gitignore` (contiene binarios y dependencias locales). En desarrollo local es habitual tener los entornos en otra ruta (p.ej. carpeta global de entornos), pero en escenarios de entrega/producción y para dockerizar suele crearse un `.venv/` dentro del repo (o dentro de la imagen) para aislar dependencias y simplificar el build.
- **docker-compose.yaml**: Orquesta el levantamiento de la API, la Demo y gestiona los volúmenes para que los datos en `data/` persistan entre reinicios.
- **pytest.ini**: Configuración centralizada de tests, definiendo rutas y marcadores para asegurar calidad de código.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de docker y testing

2.2 config/ (configuración estática)

Esta carpeta actúa como el “panel de control” del proyecto. Su objetivo es separar las **instrucciones** (el código) de las **preferencias** (la configuración). De esta forma, si queremos cambiar algo del comportamiento, no hace falta tocar el código fuente, solo editar estos archivos.

- `global_config.yaml`: Es el archivo de ajustes generales.
 - *Rutas relativas*: Le dice al programa dónde buscar los datos sin tener que escribir la dirección completa (ej: “mira en la carpeta data”).
 - *Parámetros*: Si tu IA necesita un número específico para decidir algo (como un umbral de probabilidad), se guarda aquí. Así, si mañana quieres que la IA sea más estricta, solo cambias un número en este archivo.
 - *Interruptores (Flags)*: Permite encender o apagar funciones sin borrar el código (ej: “modo_pruebas: ON/OFF”).
- `logging_config.yaml`: Es la “caja negra” o el diario de a bordo.
 - Define qué debe anotar el programa mientras funciona: ¿Queremos que guarde solo los errores graves o que anote cada paso que da?
 - Aquí configuramos si esos mensajes salen por la pantalla o se guardan en un archivo de texto para leerlos más tarde si algo falla.

2.3 data/ (persistencia y estado)

Directorio diseñado para ser montado como **Volumen de Docker**. Es necesario que esté en `.gitignore` para no subir archivos pesados a GitHub.

Actualmente no tenemos una metodología de gestión de datos ni un centro de datos en S3 o algún servidor interno. Esto implica que cada usuario tendrá que tener una copia de los datos en local. **Es MUY IMPORTANTE que todos los equipos trabajen con los mismos datos y tengan muy bien organizada la carpeta de data para que todos tengan la misma información**

- `models/`: Almacenamiento organizado por módulos.
 - `artifacts/`: Binarios finales del modelo (`.pkl`, `.pt`).
 - `metrics/`: Reportes de evaluación (`.json`, `.csv`) para monitorizar el rendimiento.
- `files/`: Pipeline de datos en sistema de archivos.
 - `raw/`: Datos crudos e inmutables.
 - `processed/`: Datos limpios listos para modelado o ingesta RAG.
 - `predictions/`: Resultados de procesos batch.
 - `splits/`: Datasets de entrenamiento/test exportados.
- `bbdd/`: Persistencia de datos estructurada.

- `sql/`: Bases de datos relacionales (SQLite local) para logs o usuarios.
- `vector/`: Índices vectoriales (ChromaDB/LanceDB) para búsquedas semánticas.
- `mlflow/`: Directorio destino para la base de datos de tracking y artefactos de experimentos (backend store y artifact store).

2.4 notebooks/ (capa de experimentación)

Todos los notebooks usados en la parte de experimentación para el desarrollo de los modelos de ML. Los modelos y resultados limpios de dichos notebooks son lo que se encapsulará en `src/` para producción.

- `EDA.ipynb`
- `experiments.ipynb`
- `evaluation.ipynb`
- ...

2.5 shared/ (librerías internas)

Contiene el código Python transversal que hace de “puente” entre la configuración (`config/`) y la lógica central (`src/`). Resuelve el *CÓMO* hacer las cosas.

Diferencia con `config`: Mientras que `config/` contiene datos estáticos (el mapa), `shared/` contiene el código ejecutable (el conductor) que usa esos datos para establecer conexiones reales.

- `database/` (Persistencia de datos):
 - `connection.py`: Implementa el patrón *Factory* para SQL. Lee la configuración y devuelve una sesión de base de datos. Si el entorno es local, conecta a SQLite (`data/bbdd/sql/`); si es producción, conecta a PostgreSQL/Cloud sin cambiar el código del resto de la app.
 - `vector_store.py`: Gestiona la conexión a la base de datos vectorial (ChromaDB/LanceDB) necesaria para el RAG. Centraliza la lógica de inicialización del cliente vectorial.
- `utils/` (Utilidades generales):
 - `llm_client.py`: Abstracción para modelos de lenguaje. Permite cambiar el motor de inteligencia (ej: de OpenAI GPT-4 a un modelo Llama 3 local en servidor propio) modificando solo la configuración. Este script se encarga de autenticar y estandarizar la llamada.
 - `security.py`: Funciones auxiliares para validación de tokens, hashing de contraseñas o gestión de claves API.

- `__init__.py`: Convierte el directorio en un paquete Python, permitiendo importaciones limpias desde cualquier parte del proyecto (ej: `from shared.database import get_db`).

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de bdd, mcp y llms

2.6 src/ (lógica central)

El núcleo del repositorio. Se divide en **módulos autónomos** con una estructura interna estandarizada. Cada módulo tiene que tener un archivo `entrypoint.py` que contiene las funciones que se pondrán disponibles vía API, es decir, las funciones que se usarán en producción (ej: `train_workflow`, `chat`, `predict`).

IMPORTANTE: solo incluir código limpio y preparado para puesta en producción.

- **Módulos de ML Clásico:**

- `entrypoint.py`
- `training/`: Lógica de entrenamiento y validación.
- `inference/`: Lógica de carga de modelos y predicción.
- `data_processing/`: ETL específico para limpiar los datos del módulo.

- **Módulos de Agentes LLM:**

- `entrypoint.py`: Fachada pública (ej.: `chat()`, `run_server()`).
- `agents/`: Implementación de agentes (roles, herramientas disponibles, políticas) de forma modular y testeable.
- `agentflows/`: Flujos/Orquestaciones de agentes (p.ej. RAG → verificación → respuesta) para separar la lógica del "workflow" del agente de la implementación de cada agente.
- `prompts/`: Los prompts del sistema se guardan aquí como código (YAML/Python), no como datos.
- `mcp_server/`: Configuración del *Model Context Protocol* y definición de Tools (herramientas) que usa el agente.
- `data_processing/`: Scripts de ingesta (ej: PDF a Embeddings) para alimentar la base de datos vectorial.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de mcp y llms

2.7 api/ (capa REST)

Capa de exposición HTTP usando **FastAPI**.

- `main.py`: Punto de entrada de la aplicación FastAPI. Define la instancia de la app, configura middlewares (CORS, logging), registra los routers y actúa como bootstrap del backend.
- `routers/`: Cada módulo de `src` tiene su propio router aquí. Se encargan de validar inputs (Pydantic) y llamar al `entrypoint` correspondiente, manteniendo la API limpia de lógica central compleja.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de FastAPI

2.8 demo/ (capa visual)

Interfaz de usuario rápida (**Streamlit**) para demos y validación interna.

- `app.py`: Gestor de navegación.
- `pages/`: Vistas individuales por caso de uso (Dashboard de ML, Chatbot de Agente), consumiendo la lógica a través de los `entrypoints` o la API.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de frontend

2.9 docker/ (contenedorización)

Configuración modular de contenedores. Al separar imágenes podemos optimizar dependencias (la API no necesita librerías de Streamlit) y mejorar seguridad, tiempos de build y tamaño final.

- `api/` (backend) y `demo/` (frontend): Tienen `Dockerfiles` separados. Esto permite dockerizar ambos componentes de forma independiente: desplegar la API como servicio backend y las demos como frontends ligeros.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de docker

2.10 scripts/ (ejecución manual / CLI)

Scripts ejecutables desde terminal para tareas de mantenimiento o ejecución batch.

- `train.py`, `predict.py`, `ingest.py`: Wrappers simples que configuran el entorno y llaman a las funciones del `entrypoint` de `src/`.

2.11 tests/ (calidad)

Estrategia de testing completa:

- `conftest.py`: Define "fixtures" (ej: bases de datos en memoria) para aislar los tests.
- `unit/`: Tests unitarios rápidos que validan lógica pura sin dependencias externas.
- `integration/`: Tests que verifican que los componentes (API, Base de datos, Modelo) funcionan correctamente en conjunto.

Estos archivos se analizarán con más detalle en sesiones técnicas específicas de testing

3. Ciclo de vida y PRs

Para mantener la estabilidad del código y facilitar la colaboración, utilizaremos una estrategia de ramas estructurada. El objetivo es aislar el desarrollo activo de la versión estable que está en producción.

3.1 Estructura de Ramas

Definimos tres niveles de estabilidad en nuestras ramas:

- **main (o prod): SOLO PRODUCCIÓN.**
 - Contiene únicamente código validado, probado y listo para ser desplegado.
 - **Prohibido** hacer commits directos (protegida).
 - Solo recibe cambios desde **dev** mediante Pull Request tras pasar una auditoría o release.
- **dev (o develop): RAMA DE INTEGRACIÓN.**
 - Es la “columna vertebral” del desarrollo diario.
 - Aquí se unen todas las funcionalidades nuevas.
 - Debe ser estable (los tests deben pasar), pero puede contener funcionalidades incompletas.
- **feature/nombre-tarea (Ramas específicas): ZONA DE TRABAJO.**
 - Ramas temporales que nacen de **dev**.
 - Aquí es donde cada desarrollador escribe su código (ej: **feature/agente-rag**, **fix/limpieza-datos**).
 - Tienen un ciclo de vida corto: se crean, se trabajan y se borran tras fusionarse.
- **archive/nombre (Ramas archivadas): HISTÓRICO.**
 - Ramas antiguas que queremos conservar (por referencia, demos, comparativas o auditoría).
 - No deben recibir desarrollo activo; si se retoma trabajo, se crea una nueva **feature/** desde **dev**.
 - Deben estar claramente etiquetadas y documentadas (por qué existen y qué versión representan).

3.2 Ciclo de Vida

(1) Inicio de Tarea:

- El desarrollador actualiza su local: `git checkout dev` y `git pull`.

- Crea su rama: `git checkout -b feature/nueva-funcionalidad`.

(2) **Desarrollo e Iteración:**

- Se realizan los cambios en local.
- Se suben a remoto (`git push`) regularmente.

(3) **Integración (PR a dev):**

- Cuando la funcionalidad está lista, se abre una **Pull Request** hacia **dev**.
- **Revisión de Código:** Otro miembro del equipo debe aprobar la PR. Esto por ahora no es obligatorio aunque puede ser recomendable según el proyecto.
- Al aprobarse, se fusiona y la rama **feature/** se borra.

Alerta (antes de abrir una PR). Mientras trabajamos en una rama **feature/**, es posible que **dev** haya recibido cambios por otro miembro del equipo. Antes de abrir la PR es necesario actualizar la rama con **dev** (merge o rebase), hacer **push** y comprobar la compatibilidad con el nuevo código. En la fase de experimentación esto no suele ser problemático (cada miembro trabaja en su notebook o módulo), pero conviene prevenir conflictos cuando se toca **src/** o **api/**.

(4) **Puesta en Producción (PR a main):**

- Ocurre solo en momentos puntuales (Releases o Checkpoints).
- Se hace una PR de **dev** hacia **main**.
- Requiere validación exhaustiva (tests de integración y aceptación).

3.3 Política de “Limpieza Trimestral” (Auditoría)

Nuestro proyecto tendrá **Checkpoints de Auditoría** cada 3 meses. En estos hitos buscamos consistencia (no perfección). Requisitos:

- **Cero ramas huérfanas:** no deben quedar ramas antiguas sin uso; las ramas **feature/** deben estar o bien integradas en **dev** o como ramas **archive/** para mantener alguna versión antigua o paralela.
- **dev actualizado:** todo el código que se considere válido debe estar integrado en **dev** y en un estado razonablemente estable/compilable.
- **Consistencia de data/ en local:** cada miembro del equipo debe asegurarse de que los datos y la estructura de la carpeta **data/** (volumen/persistencia) son los mismos en su entorno local que en el del resto del equipo.